

编译原理第七章考点复习总结

语法制导与中间代码生成

中南大学编译原理课程

2026 年 5 月

目录

1 语义分析的任务	2
2 中间代码形式	4
3 算术表达式和赋值语句的翻译	8
4 布尔表达式翻译与回填技术	10
5 控制流语句的翻译	14
6 S 属性与 L 属性定义	17
7 过程调用与数组引用的翻译	19
8 核心考点速记	20

1 语义分析的任务

7.1.1 语义分析概述 [p.3]

语义分析（Semantic Analysis）是编译器的核心阶段之一，使用属性文法（Attribute Grammar）和语法制导翻译（Syntax-Directed Translation, SDT）方法和技术来实现。语义分析通常包括以下五个方面 [p.3]：

- 1. **类型检查（Type Checking）**：验证程序中每个操作是否遵守语言的类型系统，编译器必须报告不符合类型系统的信息。
- 2. **控制流检查（Control Flow Check）**：确保控制流语句使控制转移到合法的地方（如 break 必须在循环内）。
- 3. **一致性检查（Consistency Check）**：在很多场合要求对象只能被定义一次（如标识符唯一性）。
- 4. **相关名字检查（Related Name Check）**：同一名字必须出现两次或多次（如循环名与循环开始/结束配对）。
- 5. **名字的作用域分析（Scope Analysis）**：确定标识符的可见范围。

7.1.2 类型与声明 [p.4–5]

一个类型（Type）是一组值和在这些值上的一组操作。程序设计语言中分三种类型 [p.4]：

类型	示例	说明
基本类型	int, float, double, char, bool, 枚举	原子类型，不可再分
复合类型	数组, 指针, 记录/结构/联合	由基本类型组合构成
复杂类型	链表, 栈, 队列, 树, 堆, 表	可组织为 ADT

对于声明语句，语义分析的主要任务 [p.5]：

- 收集标识符的类型等属性信息。
- 为每一个名字分配一个**相对地址（offset）**。
- **类型宽度（Width）**：从类型表达式可知该类型在运行时所需的存储单元数量。
- 名字的类型和相对地址信息保存在相应的符号表记录中。

7.1.3 变量声明语句的 SDT [p.6]

变量声明语句的语法制导定义规则，使用综合属性 type 和 width，以及变量 offset 记录下一个可用相对地址 [p.6]：

```
P → { offset = 0 } D
D → T id; { enter(id.name, T.type, offset);
           offset = offset + T.width; } D
D →
```

```

T → B { t = B.type; w = B.width; }
      C { T.type = C.type; T.width = C.width; }
T → ↑T1 { T.type = pointer(T1.type); T.width = 4; }
B → int  { B.type = int;  B.width = 4; }
B → real { B.type = real; B.width = 8; }
C →      { C.type = t; C.width = w; }
C → [num]C1 { C.type = array(num.val, C1.type);
             C.width = num.val * C1.width; }

```

关键语义函数：enter(name, type, offset) 在符号表中为名字创建记录，设置类型为 type、相对地址为 offset。

2 中间代码形式

7.2.1 中间代码概述 [p.8–10]

中间代码（Intermediate Code/Representation, IR）介乎源语言与目标代码之间，比源语言简单，比目标代码复杂 [p.9]。引入中间代码的目的 [p.10]：

- 逻辑结构清楚：便于进行与机器无关的代码优化。
- 利于移植：区分编译器的前端（Front-end）与后端（Back-end），方便针对不同目标机实现同一种语言。
- 可用于解释执行：中间代码也可作为解释器的输入。

常见中间代码形式 [p.10]：逆波兰式、三地址代码（三元式、间接三元式、四元式）、图表示（语法树/DAG）等。

7.2.2 逆波兰表示法（Reverse Polish Notation）[p.11–13]

逆波兰表示法又称后缀式（Postfix Notation），把运算量写在前面，运算符写在后面 [p.11]。

对于 K 目运算符 Q 和 K 个运算量 $e_1, e_2, \dots, e_k$ ，后缀式为：

$$e_1 e_2 \dots e_k Q$$

特点 [p.11]：只要知道每个运算符的目数，无论从哪端开始扫描，均能对后缀式进行唯一正确的分解。

计算方式 [p.12]：自左向右扫描后缀式，遇运算量则入栈，遇 k 目运算符则将栈顶 k 项取出运算，结果替换这 k 项。

优点：无需括号，便于计算机处理表达式。

示例： $(8 - 4) + 2$ 的后缀式为 $8\ 4\ -\ 2\ +$ [p.12]。

语法制导生成后缀式 [p.13]：

$E \rightarrow E(1)\ OP\ E(2) \quad \{ E.CODE := E(1).CODE \ ||\ E(2).CODE \ ||\ OP \}$

$E \rightarrow (E(1)) \quad \{ E.CODE := E(1).CODE \}$

$E \rightarrow id \quad \{ E.CODE := id \}$

【例】 $(a + b) * c - d$ 的后缀式为 $a\ b\ +\ c\ * \ d\ -$ [p.13]。

7.2.3 三地址代码（Three-Address Code, TAC）[p.14–16]

三地址代码的一般形式为 [p.14]：

$$x := y\ op\ z$$

其中 x, y, z 为名字、常量或编译时产生的临时变量； op 为运算符。

常用三地址语句 [p.16]：

类型	语句形式
二元赋值	$x := y \text{ op } z$ (op 为二元算术/逻辑算符)
一元赋值	$x := \text{op } y$ (op 为一元算符)
复制语句	$x := y$
无条件转移	goto L
条件转移	if $x \text{ relop } y$ goto L 或 if x goto L
过程调用	param x , call p, n , return y
索引赋值	$x := y[i]$, $x[i] := y$
地址/指针赋值	$x := \&y$, $x := *y$, $*x := y$

示例 [p.15]: $a := (-b + c * d) + c * d$ 的三地址代码:

```
t1 := -b
t2 := c * d
t3 := t1 + t2
t4 := c * d
t5 := t3 + t4
a := t5
```

7.2.4 三元式 (Triples) [p.17–18]

三元式的形式为 [p.17]:
(OP, ARG1, ARG2)

其中含算符、第一运算量、第二运算量三个部分。表达式及各种语句均可表示成一组三元式。

特点: 对一目运算符仅用一个运算量; 对多目算符用多个相继三元式。

示例 [p.17]: $x := -A + B * C$ 的三元式表示:

- (1) (uminus, A, -)
- (2) (*, B, C)
- (3) (+, (1), (2))
- (4) (:=, X, (3))

注意: 三元式中通过引用三元式的编号 (如 (1), (2)) 来表示中间结果, 而非临时变量。

语法制导生成三元式 [p.18]:

- (1) $E \rightarrow E(1) \text{ OP } E(2)$ { $E.val := \text{Gen}(\text{OP}, E(1).val, E(2).val)$ }
- (2) $E \rightarrow (E(1))$ { $E.val := E(1).val$ }
- (3) $E \rightarrow -E(1)$ { $E.val := \text{Gen}(\text{uminus}, E(1).val, -)$ }
- (4) $E \rightarrow i$ { $E.val := \text{Entry}(i)$ }

7.2.5 间接三元式 (Indirect Triples) [p.19–20]

除三元式表外,另设一个间接码表,按运算的先后顺序列出有关三元式在三元式表中的位置 [p.19]。

优点 [p.20]:

- 节省空间：重复的三元式只需在表中出现一次，间接码表用索引引用。
- 优化时调整顺序方便：三元式码表本身无需改动，只需重排间接码表。

示例 [p.19–20]：语句 $Y := D \uparrow (A + B)$ 与 $X := (A + B) * C$ 共享子表达式 $(A + B)$ ：

三元式表：

- (1) (+, A, B)
- (2) (*, (1), C)
- (3) (:=, X, (2))
- (4) ($\uparrow$, D, (1))
- (5) (:=, Y, (4))

间接码表：

- (1) (1) $\leftarrow$ X用
- (2) (2)
- (3) (3)
- (4) (1) $\leftarrow$ Y复用
- (5) (4)
- (6) (5)

7.2.6 四元式 (Quadruples) [p.21–22]

四元式形式为 [p.21]：

(OP, ARG1, ARG2, Result)

特点：四元式之间的联系通过临时变量实现，因此修改四元式序列容易，而修改三元式序列困难（三元式通过编号引用，牵一发而动全身）[p.21]。

示例 [p.21]： $A := -B * (C + D)$ 的四元式：

- (1) (uminus, B, -, T1)
- (2) (+, C, D, T2)
- (3) (*, T1, T2, T3)
- (4) (:=, T3, -, A)

7.2.7 四种中间代码形式对比 [p.22]

以 $A + B * (C - D) + E / (C - D) \uparrow N$ 为例 [p.22]：

逆波兰	$A B C D - * + E C D - N \uparrow / +$
四元式	(1) (-, C, D, T1) (2) (*, B, T1, T2) (3) (+, A, T2, T3) (4) (-, C, D, T4) (5) ($\uparrow$, T4, N, T5) (6) (/ , E, T5, T6) (7) (+, T3, T6, T7)
三元式	(1) (-, C, D) (2) (*, B, (1)) (3) (+, A, (2)) (4) (-, C, D) (5) ($\uparrow$, (4), N) (6) (/ , E, (5)) (7) (+, (3), (6))
间接三元式	三元式表: (1)(-,C,D) (2)(*,B,(1)) (3)(+,A,(2)) (4)($\uparrow$,(1),N) (5)(/,E,(4)) (6)(+,(3),(5)) 间接码表: (1)(2)(3)(1)(4)(5)(6)

四种形式对比总结:

特性	逆波兰式	三元式	四元式
结构	运算符前置/后置	(OP,ARG1,ARG2)	(OP,ARG1,ARG2,Result)
中间结果引用	隐式 (栈)	编号引用	临时变量名
代码移动	困难	困难 (编号关联)	容易 (变量名独立)
空间效率	高	中	低 (多一个字段)
优化友好度	低	中	高

3 算术表达式和赋值语句的翻译

7.3.1 简单算术表达式 [p.24–25]

简单算术表达式是仅含简单变量（普通变量和常数，不含数组元素及结构引用）的算术表达式 [p.24]。

核心文法 $G[A]$ [p.25]:

$$S \rightarrow id := E \quad E \rightarrow E + E \mid E * E \mid -E_1 \mid (E) \mid id$$

简单算术表达式的计值顺序与四元式出现的顺序相同，因此可以自然地翻译为四元式。

7.3.2 语义变量与语义函数 [p.27]

翻译所需的语义变量、过程及函数 [p.27]:

名称	类型	说明
E.place	语义变量	存放 E 值的变量名在符号表中的入口地址
newtemp()	函数	返回新临时变量名（依次为 T1, T2, ...）
Gen(op, arg1, arg2, result)	过程	生成一个四元式并填入四元式表
lookup(id.name)	函数	查符号表，返回入口指针或 NULL
Entry(i)	函数	返回符号 i 在符号表中的地址

7.3.3 语义子程序（翻译方案） [p.28]

文法 $G[A]$ 的语义子程序 [p.28]:

- ```
(1) S → id := E
 { p = lookup(id.name);
 if(p == NULL) error();
 else Gen(:=, E.place, -, Entry(id)); }

(2) E → E(1) + E(2)
 { E.place = newtemp();
 Gen(+, E(1).place, E(2).place, E.place); }

(3) E → E(1) * E(2)
 { E.place = newtemp();
 Gen(*, E(1).place, E(2).place, E.place); }

(4) E → -E(1)
 { E.place = newtemp();
 Gen(uminus, E(1).place, _, E.place); }

(5) E → (E(1))
```



```
{ E.place = E(1).place; }
```

(6)  $E \rightarrow id$

```
{ p = lookup(id.name);
 if(p != NULL) E.place = Entry(id);
 else error(); }
```

#### 7.3.4 SLR(1) 分析生成四元式示例 [p.29–31]

以  $A := -B * (C + D)$  为例，使用 SLR(1) 分析法（\* 优先于 +）生成四元式 [p.31]：

| 状态栈   | 符号栈    | 输入串                   | 动作/归约 | 四元式             |
|-------|--------|-----------------------|-------|-----------------|
| 0     | #      | $A := -B * (C + D)\#$ | 准备    |                 |
| 01    | #i     | $:= -B * (C + D)\#$   | 移进    |                 |
| 012   | #i:=   | $-B * (C + D)\#$      | 移进    |                 |
| 0124  | #i:=-  | $B * (C + D)\#$       | 移进    |                 |
| 01246 | #i:=-i | $*(C + D)\#$          | 移进    |                 |
| 01249 | #i:=-E | $*(C + D)\#$          | r6    | (uminus,B,-,T1) |
| 0123  | #i:=E  | $*(C + D)\#$          | r4    |                 |
| 01238 | #i:=E* | $(C + D)\#$           | 移进    |                 |
| ...   | ...    | ...                   | ...   | (+,C,D,T2)      |
| ...   | ...    | ...                   | ...   | (*,T1,T2,T3)    |
| #A    |        |                       | acc   | (:=,T3,-,A)     |

最终生成的四元式序列为：

(1) (uminus,  $B$ ,  $-$ ,  $T1$ ) (2) (+,  $C$ ,  $D$ ,  $T2$ ) (3) (\*,  $T1$ ,  $T2$ ,  $T3$ ) (4) (:=,  $T3$ ,  $-$ ,  $A$ )

#### 7.3.5 含类型转换的表达式翻译

在含混合类型运算的表达式中，需要插入类型转换四元式，如 (itr,  $X$ ,  $-$ ,  $T$ ) 表示将整数  $X$  转为实数存入  $T$ 。

## 4 布尔表达式翻译与回填技术

### 7.4.1 布尔表达式的作用与文法 [p.33]

布尔表达式在程序中两个作用 [p.33]:

- (1) 计值 (Computation): 计算出布尔值 (true/false), 如用于赋值语句。
- (2) 控制 (Control): 控制程序流程, 如 if、while 语句中的条件表达式。

布尔表达式文法 [p.33]:

$$E \rightarrow E \wedge E \mid E \vee E \mid \neg E \mid (E) \mid i \text{ rop } i \mid i$$

$$\text{rop} \rightarrow > \mid \geq \mid = \mid \leq \mid < \mid \neq$$

优先级规定 [p.33]: 关系运算符 (rop) 均相等且高于布尔运算符; 布尔运算符优先级为  $\neg > \wedge > \vee$  (递减)。

### 7.4.2 控制用布尔表达式的翻译 [p.34–35]

用于控制的布尔表达式 (如 IF E THEN S1 ELSE S2) 需要两重出口 [p.34]:

- 真出口 (True Exit): E 为真时跳转到 S1 的入口。
- 假出口 (False Exit): E 为假时跳转到 S2 的入口 (或跳过 S1)。

需要引入三种转移四元式 [p.34]:

$$\begin{aligned} (\text{Jnz}, A1, \_, P) & \text{ 若 } A1 \text{ 非零 (真) 则转到 } P \\ (\text{J}\theta, A1, A2, P) & \text{ 若 } A1 \theta A2 \text{ 成立则转到 } P (\theta \text{ 为关系运算符}) \\ (\text{J}, \_, \_, P) & \text{ 无条件转到 } P \end{aligned}$$

### 7.4.3 回填技术 (Backpatching) [p.36–38]

问题: 在自下而上分析中, 布尔表达式的真假出口往往不能在生成四元式的同时就填上, 因为目标地址尚未确定 [p.36]。

解决方案——拉链回填: 每个非终结符  $X$  赋予两个语义值 [p.36–37]:

- $X.TC$  (True Chain): 记录  $X$  对应的四元式中需要回填”真”出口的四元式地址构成的链表链首。
- $X.FC$  (False Chain): 记录需要回填”假”出口的四元式地址构成的链表链首。

当知道出口地址后, 便可顺着链回填所有待填地址。

#### 7.4.4 回填语义变量与过程 [p.38]

| 名称                    | 类型  | 说明                                            |
|-----------------------|-----|-----------------------------------------------|
| NXQ                   | 指示器 | 指向下一个将要形成的四元式地址，初值为 1，<br>每执行一次 GEN 自动加 1     |
| GEN(OP, A1, A2, addr) | 过程  | 产生四元式                                         |
| MERG(P1, P2)          | 函数  | 合并以 P1 和 P2 为链首的两条链（P1 链尾链接到 P2 链首），<br>返回新链首 |
| BACKPATCH(p, t)       | 过程  | 把以 p 为链首的链中所有四元式的第四段用 t 替换                    |

#### 7.4.5 布尔表达式的语义子程序 [p.39–40]

(1)  $E \rightarrow i$ （单个标识符）[p.39]:

```
{ E.TC := NXQ; E.FC := NXQ + 1;
 GEN(jnz, ENTRY(i), -, 0);
 GEN(j, -, -, 0); }
```

(2)  $E \rightarrow i^{(1)} \text{ rop } i^{(2)}$ （关系运算）[p.39]:

```
{ E.TC := NXQ; E.FC := NXQ + 1;
 GEN(jrop, ENTRY(i(1)), ENTRY(i(2)), 0);
 GEN(j, -, -, 0); }
```

(3)  $E \rightarrow (E^{(1)})$ （括号）[p.39]:

```
{ E.TC := E(1).TC; E.FC := E(1).FC; }
```

(4)  $E \rightarrow \neg E^{(1)}$ （逻辑非）[p.39]:

```
{ E.TC := E(1).FC; E.FC := E(1).TC; }
```

注意： $\neg$  取反直接交换 TC 和 FC。

(5)  $E^A \rightarrow E^{(1)} \wedge$ （与的”左部”翻译完）[p.40]:

```
{ BACKPATCH(E(1).TC, NXQ);
 EA.FC := E(1).FC; }
```

$\wedge$  的短路求值： $E^{(1)}$  为真时继续判断  $E^{(2)}$ ，所以回填真链使其跳转到 NXQ（ $E^{(2)}$  的起点）。假链暂时保留，因为  $E^{(1)}$  为假时整个  $E$  也为假。

(6)  $E \rightarrow E^A E^{(2)}$ （与的”右部”翻译完）[p.40]:

```
{ E.TC := E(2).TC;
 E.FC := MERG(EA.FC, E(2).FC); }
```

$E$  的真链继承自  $E^{(2)}$  的真链； $E$  的假链合并了  $E^A$  的假链（ $E^{(1)}$  为假的出口）和  $E^{(2)}$  的假链。

(7)  $E^O \rightarrow E^{(1)} \vee$ （或的”左部”翻译完）[p.40]:

```
{ BACKPATCH(E(1).FC, NXQ);
 EO.TC := E(1).TC; }
```

$\vee$  的短路求值:  $E^{(1)}$  为假时继续判断  $E^{(2)}$ , 所以回填假链使其跳转到 NXQ ( $E^{(2)}$  的起点)。

(8)  $E \rightarrow E^O E^{(2)}$  (或的”右部”翻译完) [p.40]:

```
{ E.FC := E(2).FC;
 E.TC := MERG(EO.TC, E(2).TC); }
```

$E$  的假链继承自  $E^{(2)}$  的假链;  $E$  的真链合并了  $E^O$  的真链 ( $E^{(1)}$  为真的出口) 和  $E^{(2)}$  的真链。

#### 7.4.6 拉链回填图解

| 操作            | $\wedge$ (与)                        | $\vee$ (或)                          |
|---------------|-------------------------------------|-------------------------------------|
| $E^{(1)}$ 为真时 | 回填真链 $\rightarrow$ 跳转到 $E^{(2)}$ 起点 | 真链保留 (整个 $E$ 为真)                    |
| $E^{(1)}$ 为假时 | 假链保留 (整个 $E$ 为假)                    | 回填假链 $\rightarrow$ 跳转到 $E^{(2)}$ 起点 |
| $E$ 的真出口      | $E^{(2)}$ 的真链                       | 合并 $E^{(1)}$ 和 $E^{(2)}$ 的真链        |
| $E$ 的假出口      | 合并 $E^{(1)}$ 和 $E^{(2)}$ 的假链        | $E^{(2)}$ 的假链                       |

#### 7.4.7 完整示例: $P < Q \vee R < S \wedge U > V$ [p.41–43]

布尔表达式  $P < Q \vee R < S \wedge U > V$  的四元式生成过程 ( $\wedge$  优先级高于  $\vee$ ) [p.41–42]:

| 步骤                            | 归约动作                                 | 生成四元式                                             |
|-------------------------------|--------------------------------------|---------------------------------------------------|
| 归约 $P < Q$ 为 $E$              | (2) $E \rightarrow i \text{ rop } i$ | (1) (j<,P,Q,0), (2) (j,-,-,0)<br>E.TC=1, E.FC=2   |
| 遇到 $\vee$ , 归约 (7)            | $E^O \rightarrow E^{(1)} \vee$       | 回填 E.FC(即 2): (2)(j,-,-,3)<br>EO.TC = 1 (=E.TC)   |
| 归约 $R < S$ 为 $E$              | (2) $E \rightarrow i \text{ rop } i$ | (3) (j<,R,S,0), (4) (j,-,-,0)<br>E.TC=3, E.FC=4   |
| 遇到 $\wedge$ , 归约 (5)          | $E^A \rightarrow E^{(1)} \wedge$     | 回填 E.TC(即 3): (3)(j<,R,S,5)<br>EA.FC = 4 (=E.FC)  |
| 归约 $U > V$ 为 $E$              | (2) $E \rightarrow i \text{ rop } i$ | (5) (j>,U,V,0), (6) (j,-,-,0)<br>E.TC=5, E.FC=6   |
| 归约 (6): $E \rightarrow E^A E$ | 合并                                   | E.TC=5, E.FC=MERG(4,6)=6<br>回填 (6): (6)(j,-,-,4)  |
| 归约 (8): $E \rightarrow E^O E$ | 合并                                   | E.TC=MERG(1,5)=5, E.FC=6<br>回填 (5): (5)(j>,U,V,1) |

最终 [p.43]:  $E.TC = 5$  (链: (5)(j>,U,V,1) $\rightarrow$ (1)(j<,P,Q,0)),  $E.FC = 6$  (链: (6)(j,-,-,4) $\rightarrow$ (4)(j,-,-,0))。

#### 7.4.8 IF-THEN-ELSE 完整翻译示例 [p.44–45]

例: if  $P < Q \vee R < S \wedge U > V$  then  $I := I + I$  else  $I := I * I$  [p.44–45]

布尔表达式部分如 7.4.7 节所述; 赋值语句部分:

- $I := I + I$  的四元式 (真分支) [p.44]:

(7) (+,  $I$ ,  $I$ , T1) (8) (:=, T1, -,  $I$ ) (9) (j, -, -, 12)

- $I := I * I$  的四元式（假分支）[p.45]:

$$(10) (*, I, I, T2) \quad (11) (:=, T2, -, I)$$

最终全部四元式（回填后）[p.45]:

|     |               |      |                |
|-----|---------------|------|----------------|
| (1) | (j<, P, Q, 7) | (7)  | (+, I, I, T1)  |
| (2) | (j, -, -, 3)  | (8)  | (:=, T1, -, I) |
| (3) | (j<, R, S, 5) | (9)  | (j, -, -, 12)  |
| (4) | (j, -, -, 10) | (10) | (*, I, I, T2)  |
| (5) | (j>, U, V, 7) | (11) | (:=, T2, -, I) |
| (6) | (j, -, -, 10) | (12) |                |

执行流程:

- $P < Q$  真  $\rightarrow$  (1) 跳转到 (7)（真分支入口）。
- $P < Q$  假  $\rightarrow$  (2) 转到 (3) 继续判断  $R < S$ 。
- $R < S$  真  $\rightarrow$  (3) 跳转到 (5) 判断  $U > V$ 。
- $R < S$  假  $\rightarrow$  (4) 跳转到 (10)（假分支入口）——短路求值。
- $U > V$  真  $\rightarrow$  (5) 回填后跳转到 (7)（真分支）。
- $U > V$  假  $\rightarrow$  (6) 跳转到 (10)（假分支）。
- 真分支执行完毕后 (9) 无条件跳转到 (12)（跳过假分支）。

## 5 控制流语句的翻译

### 7.5.1 S 属性与 L 属性定义

在语法制导翻译中，属性文法的两类重要定义：

**S 属性定义 (S-Attributed Definition):**

- 仅使用综合属性 (Synthesized Attributes)。
- 综合属性：父节点的属性值由子节点的属性值计算得到。
- 适合自下而上 (Bottom-Up) 分析，可在归约时计算。
- 四元式生成中的  $E.place$ ,  $E.TC$ ,  $E.FC$  均为综合属性。

**L 属性定义 (L-Attributed Definition):**

- 可以使用综合属性和继承属性 (Inherited Attributes)。
- 继承属性的约束：子节点的继承属性只能依赖父节点的继承属性或左边兄弟节点的任意属性。
- 适合自上而下 (Top-Down) 和自下而上的分析。
- 例：声明语句中 `offset` 是继承属性——每个变量的地址依赖前面变量累积的偏移量。

### 7.5.2 翻译方案 (Translation Scheme)

翻译方案是在文法产生式中嵌入语义动作（用花括号括起来）的上下文无关文法。与语法制导定义的区别：翻译方案显式规定了语义动作的执行顺序。

建立翻译方案的原则：

- S 属性定义：在产生式右端的末尾嵌入所有语义动作（归约时统一执行）。
- L 属性定义（自顶向下）：语义动作可嵌入产生式右端任意位置，但必须保证动作所需的属性值已计算完毕。
- 继承属性通过复写规则 (Copy Rule) 在产生式左边符号展开前传递给右边符号。

### 7.5.3 IF-THEN-ELSE 语句翻译

文法： $S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

采用回填技术的翻译方案：

```
S → if E then M1 S1 N else M2 S2
{ BACKPATCH(E.TC, M1.quad); // 真出口→S1入口
 BACKPATCH(E.FC, M2.quad); // 假出口→S2入口
 S.nextlist = MERG(S1.nextlist, N.nextlist, S2.nextlist); }
```

其中  $M \rightarrow \varepsilon$  (标记非终结符，记录下一条四元式地址)：

```
M → { M.quad := NXQ; }
```

$N \rightarrow \varepsilon$  (在  $S_1$  之后生成 goto 跳过 else 分支):

```
N → { N.nextlist := makelist(NXQ);
 GEN(j, -, -, 0); }
```

#### 7.5.4 WHILE 循环语句翻译

文法:  $S \rightarrow \text{while } E \text{ do } S_1$

翻译方案:

```
S → while M1 E do M2 S1
{ BACKPATCH(S1.nextlist, M1.quad); // S1结束后回跳到条件判断
 BACKPATCH(E.TC, M2.quad); // 真出口→循环体S1入口
 S.nextlist = E.FC; // 假出口→循环后语句
 GEN(j, -, -, M1.quad); // 生成回跳指令
}
```

其中:

```
M1 → { M1.quad := NXQ; } // 记录条件E的起始地址 (循环回跳目标)
M2 → { M2.quad := NXQ; } // 记录S1的起始地址
```

执行流程: 先计算  $E$ , 若真跳入  $S_1$ ,  $S_1$  执行完后无条件跳回  $M_1$  (重新判断条件); 若假退出循环。

#### 7.5.5 FOR 循环语句翻译

文法:  $S \rightarrow \text{for } (S_1; E; S_2) S_3$

等价于:

```
S1; // 初始化
L1: if not E goto L2; // 条件判断
S3; // 循环体
L0: S2; // 增量
goto L1; // 回跳
L2: // 出口
```

四元式翻译模式 (回填版本):

```
(1) S1 的代码
(2) (jnz, E.place, -, (4)) // 真→循环体
(3) (j, -, -, 出口) // 假→退出
(4) S3 的代码
(5) S2 的代码
(6) (j, -, -, (2)) // 回跳条件判断
(7) // 出口
```

### 7.5.6 SWITCH-CASE 语句翻译

翻译策略：

1. 计算开关表达式值，存入临时变量  $t$ 。
2. 对每个 case 值  $v_i$ ，生成条件跳转  $(j =, t, v_i, L_i)$ 。
3. 生成默认跳转  $(j, -, -, L_{default})$ （对应 default 或无匹配时的出口）。
4. 每个 case 分支末尾生成无条件跳转到 switch 出口（除非有 break）。



## 6 S 属性与 L 属性定义

### 7.6.1 属性文法基础

**属性 (Attribute):** 与文法符号相关联的量，描述符号的语义特征。

**属性分类:**

| 类型                 | 计算方向                     | 适用分析      |
|--------------------|--------------------------|-----------|
| 综合属性 (Synthesized) | 子节点 $\rightarrow$ 父节点    | 自下而上      |
| 继承属性 (Inherited)   | 父节点/兄弟 $\rightarrow$ 子节点 | 自顶向下/自下而上 |

**语义规则 (Semantic Rule):** 为每个产生式定义的属性计算规则。语义规则定义了属性之间的依赖关系。

### 7.6.2 S 属性定义的特性

- 每个产生式的语义规则仅计算产生式左部符号的综合属性。
- 属性值依赖产生式右部符号的综合属性。
- 可用自下而上分析器（如 LR 分析器）自然地实现：归约时调用语义子程序。
- 前面三地址代码生成中的属性 `E.place`, `E.TC`, `E.FC`, `B.type`, `B.width` 都是综合属性。

### 7.6.3 L 属性定义的特性

- 对每个产生式  $A \rightarrow X_1X_2 \dots X_n$ ，每个语义规则计算：
  - $A$  的综合属性，或
  - $X_j$  ( $1 \leq j \leq n$ ) 的继承属性。
- $X_j$  的继承属性只能依赖：
  - $A$  的继承属性，或
  - $X_1, X_2, \dots, X_{j-1}$ （左边兄弟）的任意属性。
- 关键约束：不得依赖右边兄弟的属性。
- 适合深度优先遍历语法制导翻译。

7.6.4 两类定义的对比

| 特性   | S 属性定义        | L 属性定义              |
|------|---------------|---------------------|
| 属性类型 | 仅综合属性         | 综合属性 + 受约束的继承属性     |
| 实现方式 | 自下而上 (LR 分析器) | 自顶向下 (LL 分析器) 或自下而上 |
| 计算时机 | 归约时           | 深度优先遍历时             |
| 典型应用 | 表达式求值、中间代码生成  | 声明处理、类型信息传递         |
| 复杂度  | 低             | 中                   |
| 表达能力 | 较弱            | 较强                  |

**重要结论：**任何 S 属性定义一定是 L 属性定义，但反之不成立 (L 属性定义比 S 属性定义表达力更强)。

## 7 过程调用与数组引用的翻译

### 7.7.1 过程调用的四元式翻译

过程调用的翻译涉及参数传递和调用两个阶段 [p.16]:

```
param x // 将参数x压入参数区
call p, n // 调用过程p, 共有n个参数
return y // 返回, 返回值在y中
```

示例: 调用 `foo(a+b, c)` 的四元式:

```
T1 := a + b
param T1
param c
call foo, 2
```

### 7.7.2 数组元素引用的翻译

**数组元素地址计算:** 对于数组  $A[l_1 : u_1, l_2 : u_2, \dots, l_k : u_k]$ , 元素  $A[i_1, i_2, \dots, i_k]$  的地址 (行主序) 为:

$$\text{base} + ((\dots((i_1 - l_1) \times d_2 + (i_2 - l_2)) \times d_3 + \dots) \times d_k + (i_k - l_k)) \times w$$

其中  $d_j = u_j - l_j + 1$  (第  $j$  维的元素个数),  $w$  为元素宽度。

可重写为递推形式 (减少乘法次数):

$$\text{addr} = \text{base} - C + \sum_{j=1}^k i_j \times n_j$$

$$n_k = w$$

$$n_{j-1} = n_j \times d_j \quad (j = k, \dots, 2)$$

$$C = \sum_{j=1}^k l_j \times n_j$$

其中  $C$  可在编译时计算 (因为下界和维长已知), 运行时只需计算  $\sum i_j \times n_j$ 。

**三地址代码翻译:**

- 赋值  $X := A[I_1, I_2, \dots, I_k]$ : 计算地址, 用索引加载四元式。
- 赋值  $A[I_1, I_2, \dots, I_k] := X$ : 计算地址, 用索引存储四元式。

## 8 核心考点速记

### 8.1 关键翻译模式/语义规则速查

| 知识点                                     | 核心规则/模式                                                                                                       |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------|
| 后缀式生成                                   | $E \rightarrow E^{(1)} \text{ op } E^{(2)}$ : $E.CODE = E(1).CODE \    \ E(2).CODE \    \ \text{op}$ [p.13]   |
| 四元式生成                                   | <code>newtemp()</code> 产生临时变量; <code>Gen(op, arg1, arg2, result)</code> 生成四元式 [p.27]                          |
| 声明 SDT                                  | <code>offset</code> 累积偏移; <code>T.type / T.width</code> 向上综合传递 [p.6]                                          |
| 布尔表达式: $E \rightarrow i$                | $TC = NXQ$ ; $FC = NXQ + 1$ ; <code>Gen(jnz, i, 0)</code> ; <code>Gen(j, 0, 0)</code> [p.39]                  |
| 布尔表达式: $E \rightarrow i \text{ rop } i$ | $TC = NXQ$ ; $FC = NXQ + 1$ ; <code>Gen(jrop, i1, i2, 0)</code> ; <code>Gen(j, 0, 0)</code> [p.39]            |
| 布尔表达式: $\neg E$                         | $TC$ 与 $FC$ 互换 [p.39]                                                                                         |
| 布尔表达式: $E^{(1)} \wedge$                 | <code>Backpatch(E(1).TC, NXQ)</code> ; $EA.FC = E(1).FC$ [p.40]                                               |
| 布尔表达式: $E^A E^{(2)}$                    | $E.TC = E(2).TC$ ; $E.FC = \text{MERG}(EA.FC, E(2).FC)$ [p.40]                                                |
| 布尔表达式: $E^{(1)} \vee$                   | <code>Backpatch(E(1).FC, NXQ)</code> ; $EO.TC = E(1).TC$ [p.40]                                               |
| 布尔表达式: $E^O E^{(2)}$                    | $E.FC = E(2).FC$ ; $E.TC = \text{MERG}(EO.TC, E(2).TC)$ [p.40]                                                |
| 回填核心操作                                  | <code>MERG(p1, p2)</code> : 链合并; <code>BACKPATCH(p, t)</code> : 链回填 [p.38]                                    |
| IF-THEN-ELSE                            | 真出口 $\rightarrow S1$ 入口, 假出口 $\rightarrow S2$ 入口, $S1$ 后无条件跳转跳过 $S2$                                          |
| WHILE-DO                                | 回跳到条件判断入口 ( <code>M1.quad</code> ), 假出口为循环后续语句                                                                |
| 变量声明                                    | $P \rightarrow \{offset = 0\}D$ ; 每个变量 <code>enter</code> 后 <code>offset</code> 累加 <code>T.width</code> [p.6] |
| 类型转换                                    | 混合类型运算需插入类型转换四元式, 如 $(itr, int\_val, -, real\_temp)$                                                          |

8.2 四种中间代码形式对比表

| 特性    | 逆波兰式（后缀式）  | 三元式                | 四元式                      |
|-------|------------|--------------------|--------------------------|
| 形式    | 运算量在前，算符在后 | (OP, ARG1, ARG2)   | (OP, ARG1, ARG2, Result) |
| 中间结果  | 隐式（栈中）     | 三元式编号引用（如 (1),(2)） | 显式临时变量（如 T1,T2）          |
| 代码移动性 | 差（依赖栈序）    | 差（编号关联，牵一发发动全身）    | 好（变量名独立）                 |
| 优化方便度 | 困难         | 较困难                | 方便                       |
| 空间效率  | 最高         | 中（3 字段）            | 较低（4 字段）                 |
| 典型用途  | 表达式求值、栈式计算 | 代码生成中间阶段           | 优化和代码生成                  |
| 间接三元式 | —          | 加间接码表；复用三元式省空间；    | 重排序只需改间接码表（三元式不动）        |

8.3 回填技术核心要点

- 1. 为什么需要回填：自下而上分析时，转移目标地址在生成跳转指令时尚未确定。
- 2. 拉链机制：待回填的四元式通过链指针串联起来（TC 真链，FC 假链）。
- 3. 短路求值： $\wedge$  遇假短路（ $E_1$  假时直接假出口）； $\vee$  遇真短路（ $E_1$  真时直接真出口）。
- 4. 回填时机：当获得目标地址后，调用 BACKPATCH(链首, 目标地址)。
- 5. 链合并： $\wedge$  结果是合并  $E_1$  假链和  $E_2$  假链； $\vee$  结果是合并  $E_1$  真链和  $E_2$  真链。
- 6. 最终回填：整个布尔表达式分析完后，由外层语句（if/while）提供真正的真假出口地址进行回填 [p.43-45]。

8.4 重点记忆公式

- (1) 后缀式翻译:  $E.CODE := E_1.CODE \parallel E_2.CODE \parallel OP$  [p.13]
- (2) 四元式生成:  $E.place := newtemp(); Gen(op, E_1.place, E_2.place, E.place)$  [p.28]
- (3) 逻辑非交换律:  $\neg E \implies E.TC \leftrightarrow E.FC$  [p.39]
- (4)  $\wedge$  回填真链:  $BACKPATCH(E_1.TC, NXQ)$  [p.40]
- (5)  $\vee$  回填假链:  $BACKPATCH(E_1.FC, NXQ)$  [p.40]

(6) 声明式 offset 更新:  $offset := offset + T.width$  [p.6]

(7) 数组地址:  $base - C + \sum i_j \times n_j$ , 其中  $C$  编译时可计算。

## 8.5 常见考题类型

1. 后缀式/逆波兰表示: 给定中缀表达式, 写出其后缀式 (参考 [p.46] 作业 1)。
2. 三地址代码生成: 给定赋值语句, 写出其三元式或四元式序列 (参考 [p.47] 作业 2)。
3. 布尔表达式四元式生成过程: 完整展示拉链回填过程, 写出每一步的四元式和 TC/FC 链变化 (参考 [p.47] 作业 3)。
4. IF/WHILE 语句完整四元式序列: 结合布尔表达式翻译和回填, 写出最终四元式列表。
5. 四种中间代码对比: 逆波兰、三元式、间接三元式、四元式的形式、特点和适用场景。
6. 语法制导定义: 给定文法, 写出语义规则, 区分 S 属性和 L 属性。
7. 声明语句处理: 给出类型宽度计算和符号表填写过程。